

字节跳动最爱考的 64 道算法题

五角钱的程序员 今天

以下文章来源于图雀社区，作者一只图雀



图雀社区

汇集精彩的实战教程，关注回复「交流」拉你进学习交流群，与爱好技术的小伙伴共同成...

缘起

现在大厂面试中，算法题几乎为必考项，且近几年频现 LeetCode 真题，此篇为拿到字节、腾讯、京东 Offer 的笔者本人在准备面试过程中亲自刷过以及遇到过高频算法题。文章内容会分模块整理，对于笔者在面试过程中遇到的真题，会给予着重 【🔥】标出。

同时，可以毫不客气的说，如果你准备时间有限，又想追求算法题准备效率最大化，那么你只需要按照大纲把下面的题目刷完，并把代码烂熟于心，就几乎可以应对 90% 的面试算法考题了。

整理这篇内容的目的一个是笔者在之前准备面试时的一点积累，而它确实也帮助笔者在面试算法题中过关斩将，同时呢，也希望能够在金三银四给予拼搏的你，一点点帮助就好！



文末有福利：) 🐱

本篇内容包括如下模块：

- 高频算法题系列：链表
- 【🔥】【有真题】高频算法题系列：字符串
- 【🔥】【有真题】高频算法题系列：数组问题
- 高频算法题系列：二叉树
- 【🔥】高频算法题系列：排序算法
- 【🔥】高频算法题系列：二分查找
- 【🔥】高频算法题系列：动态规划

- 高频算法题系列：BFS
- 【🔥】高频算法题系列：栈
- 【🔥】高频算法题系列：DFS
- 【🔥】高频算法题系列：回溯算法

其中标🔥的部分代表非常高频的考题，其中不乏笔者遇到的原题。其中对于每一类，首先会列出包含的考题，然后针对每一道考题会给出难度、考察知识点、是否是面试真题，在每道题详细介绍时，还会给出每道题的 LeetCode 链接，帮助读者理解题意，以及能够进行实际的测验，还可以观看其他人的答案，更好的帮助准备。

高频算法题系列：链表

笔者遇到的高频链表题主要包含这几道：

- 通过快慢指针寻找链表中点 【简单】
- 通过链表的后续遍历判断回文链表问题 【简单】
- 链表的反向输出 【简单】
- 合并 K 个升序链表 【困难】
- K个一组翻转链表 【困难】
- 环形链表 【简单】
- 排序链表 【中等】
- 相交链表 【简单】

寻找链表中点

题解

通过快慢指针寻找链表中点

```
/**
 *
 */

function findCenter(head) {
    let slower = head, faster = head;
    while (faster && faster.next != null) {
        slower = slower.next;
```

```

        slower = slower.next;
        faster = faster.next.next;
    }
    // 如果 faster 不等于 null, 说明是奇数个, slower 再移动一格
    if (faster != null) {
        slower = slower.next;
    }
    return slower;
}

```

前序遍历判断回文链表

👉 **【LeetCode 直通车】：234 回文链表（简单）** ^[1]

题解1

利用链表的后续遍历，使用函数调用栈作为后序遍历栈，来判断是否回文

```

/**
 *
 */
var isPalindrome = function(head) {
    let left = head;
    function traverse(right) {
        if (right == null) return true;
        let res = traverse(right.next);
        res = res && (right.val === left.val);
        left = left.next;
        return res;
    }
    return traverse(head);
};

```

题解2

通过 快、慢指针找链表中点，然后反转链表，比较两个链表两侧是否相等，来判断是否是回文链表

```

/**
 *

```

```
*/  
  
var isPalindrome = function(head) {  
    // 反转 slower 链表  
    let right = reverse(findCenter(head));  
    let left = head;  
    // 开始比较  
    while (right != null) {  
        if (left.val !== right.val) {  
            return false;  
        }  
        left = left.next;  
        right = right.next;  
    }  
    return true;  
}  
  
function findCenter(head) {  
    let slower = head, faster = head;  
    while (faster && faster.next != null) {  
        slower = slower.next;  
        faster = faster.next.next;  
    }  
    // 如果 faster 不等于 null, 说明是奇数个, slower 再移动一格  
    if (faster != null) {  
        slower = slower.next;  
    }  
    return slower;  
}  
  
function reverse(head) {  
    let prev = null, cur = head, nxt = head;  
    while (cur != null) {  
        nxt = cur.next;  
        cur.next = prev;  
        prev = cur;  
        cur = nxt;  
    }  
    return prev;  
}
```

反转链表

👉 **【LeetCode 直通车】：206 反转链表（简单）** [2]

题解

```

/**
 * Definition for singly-linked list.
 * function ListNode(val) {
 *     this.val = val;
 *     this.next = null;
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var reverseList = function(head) {
    if (head == null || head.next == null) return head;
    let last = reverseList(head.next);
    head.next.next = head;
    head.next = null;
    return last;
};

```

合并K个升序链表

👉 **【LeetCode 直通车】：23 合并K个升序链表（困难） [3]**

题解

```

/**
 * Definition for singly-linked list.
 * function ListNode(val) {
 *     this.val = val;
 *     this.next = null;
 * }
 */
/**
 * @param {ListNode[]} lists
 * @return {ListNode}
 */
var mergeKLists = function(lists) {
    if (lists.length === 0) return null;
    return mergeArr(lists);
};

```

```
};  
function mergeArr(lists) {  
    if (lists.length <= 1) return lists[0];  
    let index = Math.floor(lists.length / 2);  
    const left = mergeArr(lists.slice(0, index));  
    const right = mergeArr(lists.slice(index));  
    return merge(left, right);  
}  
function merge(l1, l2) {  
    if (l1 == null && l2 == null) return null;  
    if (l1 != null && l2 == null) return l1;  
    if (l1 == null && l2 != null) return l2;  
    let newHead = null, head = null;  
    while (l1 != null && l2 != null) {  
        if (l1.val < l2.val) {  
            if (!head) {  
                newHead = l1;  
                head = l1;  
            } else {  
                newHead.next = l1;  
                newHead = newHead.next;  
            }  
            l1 = l1.next;  
        } else {  
            if (!head) {  
                newHead = l2;  
                head = l2;  
            } else {  
                newHead.next = l2;  
                newHead = newHead.next;  
            }  
            l2 = l2.next;  
        }  
    }  
    newHead.next = l1 ? l1 : l2;  
    return head;  
}
```

K 个一组翻转链表

👉 **【LeetCode 直通车】：25 K 个一组翻转链表（困难）** [4]

题解

```

/**
 * Definition for singly-linked list.
 * function ListNode(val) {
 *     this.val = val;
 *     this.next = null;
 * }
 */

/**
 * @param {ListNode} head
 * @param {number} k
 * @return {ListNode}
 */
var reverseKGroup = function(head, k) {
    let a = head, b = head;
    for (let i = 0; i < k; i++) {
        if (b == null) return head;
        b = b.next;
    }
    const newHead = reverse(a, b);
    a.next = reverseKGroup(b, k);
    return newHead;
};

function reverse(a, b) {
    let prev = null, cur = a, nxt = a;
    while (cur != b) {
        nxt = cur.next;
        cur.next = prev;
        prev = cur;
        cur = nxt;
    }
    return prev;
}

```

环形链表

👉 **【LeetCode 直通车】：141 环形链表（简单） [5]**

题解

```

/**
 * Definition for singly-linked list.

```

```

* function ListNode(val) {
*     this.val = val;
*     this.next = null;
* }
*/
/**
 * @param {ListNode} head
 * @return {boolean}
 */
var hasCycle = function(head) {
    if (head == null || head.next == null) return false;
    let slower = head, faster = head;
    while (faster != null && faster.next != null) {
        slower = slower.next;
        faster = faster.next.next;
        if (slower === faster) return true;
    }
    return false;
};

```

排序链表

👉 **【LeetCode 直通车】：148 排序链表（中等） [6]**

题解

```

/**
 * Definition for singly-linked list.
 * function ListNode(val) {
 *     this.val = val;
 *     this.next = null;
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var sortList = function(head) {
    if (head == null) return null;
    let newHead = head;

```

```
    return mergeSort(head);
};
function mergeSort(head) {
    if (head.next !== null) {
        let slower = getCenter(head);
        let nxt = slower.next;
        slower.next = null;
        console.log(head, slower, nxt);
        const left = mergeSort(head);
        const right = mergeSort(nxt);
        head = merge(left, right);
    }
    return head;
}
function merge(left, right) {
    let newHead = null, head = null;
    while (left !== null && right !== null) {
        if (left.val < right.val) {
            if (!head) {
                newHead = left;
                head = left;
            } else {
                newHead.next = left;
                newHead = newHead.next;
            }
            left = left.next;
        } else {
            if (!head) {
                newHead = right;
                head = right;
            } else {
                newHead.next = right;
                newHead = newHead.next;
            }
            right = right.next;
        }
    }
    newHead.next = left ? left : right;
    return head;
}
function getCenter(head) {
    let slower = head, faster = head.next;
    while (faster !== null && faster.next !== null) {
        slower = slower.next;
        faster = faster.next.next;
    }
}
```

```
    return slower;
}
```

相交链表

👉 **【LeetCode 直通车】：160 相交链表（简单） [7]**

题解

```
/**
 * Definition for singly-linked list.
 * function ListNode(val) {
 *     this.val = val;
 *     this.next = null;
 * }
 */
/**
 * @param {ListNode} headA
 * @param {ListNode} headB
 * @return {ListNode}
 */
var getIntersectionNode = function(headA, headB) {
    let lastHeadA = null;
    let lastHeadB = null;
    let originHeadA = headA;
    let originHeadB = headB;
    if (!headA || !headB) {
        return null;
    }
    while (true) {
        if (headB == headA) {
            return headB;
        }
        if (headA && headA.next == null) {
            lastHeadA = headA;
            headA = originHeadB;
        } else {
            headA = headA.next;
        }
        if (headB && headB.next == null) {
            lastHeadB = headB;
        }
    }
}
```

```

        headB = originHeadA;
    } else {
        headB = headB.next;
    }
    if (lastHeadA && lastHeadB && lastHeadA != lastHeadB) {
        return null;
    }
}
return null;
};

```

【🔥】高频算法题系列：字符串

主要有以下几类高频考题：

- 最长回文子串 【中等】 【双指针】 【面试真题】
- 最长公共前缀 【简单】 【双指针】
- 无重复字符的最长子串 【中等】 【双指针】
- 最小覆盖子串 【困难】 【滑动窗口】 【面试真题】

【面试真题】最长回文子串【双指针】

👉 【LeetCode 直通车】：5 最长回文子串（中等） [8]

题解

```

/**
 * @param {string} s
 * @return {string}
 */
var longestPalindrome = function(s) {
    if (s.length === 1) return s;
    let maxRes = 0, maxStr = '';
    for (let i = 0; i < s.length; i++) {
        let str1 = palindrome(s, i, i);
        let str2 = palindrome(s, i, i + 1);
        if (str1.length > maxRes) {
            maxStr = str1;
            maxRes = str1.length;
        }
        if (str2.length > maxRes) {
            maxStr = str2;
            maxRes = str2.length;
        }
    }
    return maxStr;
};

```

```

        maxRes = str1.length;
    }
    if (str2.length > maxRes) {
        maxStr = str2;
        maxRes = str2.length;
    }
}
return maxStr;
};

function palindrome(s, l, r) {
    while (l >= 0 && r < s.length && s[l] === s[r]) {
        l--;
        r++;
    }
    return s.slice(l + 1, r);
}

```

最长公共前缀【双指针】

👉 **【LeetCode 直通车】：14 最长公共前缀（简单）** [9]

题解

```

/**
 * @param {string[]} strs
 * @return {string}
 */
var longestCommonPrefix = function(strs) {
    if (strs.length === 0) return "";
    let first = strs[0];
    if (first === "") return "";
    let minLen = Number.MAX_SAFE_INTEGER;
    for (let i = 1; i < strs.length; i++) {
        const len = twoStrLongestCommonPrefix(first, strs[i]);
        minLen = Math.min(len, minLen);
    }
    return first.slice(0, minLen);
};

function twoStrLongestCommonPrefix (s, t) {
    let i = 0, j = 0;
    let cnt = 0;
    while (i < s.length && j < t.length) {
        console.log(s[i], t[j], cnt)
    }
}

```

```
        if (s[i] === t[j]) {
            cnt++;
        } else {
            return cnt;
        }
        i++;
        j++;
    }
    return cnt;
}
```

无重复字符的最长子串【双指针】

👉 **【LeetCode 直通车】：3 无重复字符的最长子串（中等） [10]**

题解

```
/**
 * @param {string} s
 * @return {number}
 */
var lengthOfLongestSubstring = function(s) {
    let window = {};
    let left = 0, right = 0;
    let maxLen = 0, maxStr = '';
    while (right < s.length) {
        let c = s[right];
        right++;
        if (window[c]) window[c]++;
        else window[c] = 1
        while (window[c] > 1) {
            let d = s[left];
            left++;
            window[d]--;
        }
        if (maxLen < right - left) {
            maxLen = right - left;
        }
    }
    return maxLen;
};
```

【面试真题】最小覆盖子串【滑动窗口】

👉 【LeetCode 直通车】：76 最小覆盖子串（困难） [11]

题解

```
/**
 * @param {string} s
 * @param {string} t
 * @return {string}
 */
var minWindow = function(s, t) {
    let need = {}, window = {};
    for (let c of t) {
        if (!need[c]) need[c] = 1;
        else need[c]++;
    }
    let left = 0, right = 0;
    let valid = 0, len = Object.keys(need).length;
    let minLen = s.length + 1, minStr = '';
    while (right < s.length) {
        const d = s[right];
        right++;
        if (!window[d]) window[d] = 1;
        else window[d]++;
        if (need[d] && need[d] === window[d]) {
            valid++;
        }
        console.log('left - right', left, right);
        while (valid === len) {
            if (right - left < minLen) {
                minLen = right - left;
                minStr = s.slice(left, right);
            }
            console.log('minStr', minStr);
            let c = s[left];
            left++;
            window[c]--;
            if (need[c] && window[c] < need[c]) {
                valid--;
            }
        }
    }
}
```

```
    }  
    return minStr;  
};
```

【🔥】高频算法题系列：数组问题

主要有几类高频考题：

- 俄罗斯套娃信封问题【困难】 【排序+最长上升子序列】 【面试真题】
- 最长连续递增序列 【简单】 【双指针】
- 最长连续序列【困难】 【哈希表】
- 盛最多水的容器【困难】 【面试真题】
- 寻找两个正序数组的中位数【困难】 【双指针】
- 删除有序数组中的重复项【简单】 【快慢指针】
- 和为K的子数组【中等】 【哈希表】
- nSum 问题【系列】 【简单】 【哈希表】
- 接雨水【困难】 【暴力+备忘录优化】 【面试真题】
- 跳跃游戏【系列】 【中等】 【贪心算法】

【面试真题】俄罗斯套娃信封问题【排序+最长上升子序列】

👉 【LeetCode 直通车】：354 俄罗斯套娃信封问题（困难） [12]

题解

```
/**  
 * @param {number[][]} envelopes  
 * @return {number}  
 */  
var maxEnvelopes = function(envelopes) {  
    if (envelopes.length === 1) return 1;  
    envelopes.sort((a, b) => {  
        if (a[0] !== b[0]) return a[0] - b[0];  
        else return b[1] - a[1];  
    });  
    let LISArr = [];  
    for (let [key, value] of envelopes) {
```

```

    for ([key, value] of envelopes) {
        LISArr.push(value);
    }
    console.log( LISArr);
    return LIS(LISArr);
};
function LIS(arr) {
    let dp = [];
    let maxAns = 0;
    for (let i = 0; i < arr.length; i++) {
        dp[i] = 1;
    }
    for (let i = 1; i < arr.length; i++) {
        for (let j = i; j >= 0; j--) {
            if (arr[i] > arr[j]) {
                dp[i] = Math.max(dp[i], dp[j] + 1)
            }
            maxAns = Math.max(maxAns, dp[i]);
        }
    }
    return maxAns;
}

```

最长连续递增序列【快慢指针】

👉 【LeetCode 直通车】：674 最长连续递增序列（简单） [13]

题解

```

/**
 * @param {number[]} nums
 * @return {number}
 */
var findLengthOfLCIS = function(nums) {
    if (nums.length === 0) return 0;
    const n = nums.length;
    let left = 0, right = 1;
    let globalMaxLen = 1, maxLen = 1;
    while (right < n) {
        if (nums[right] > nums[left]) maxLen++;
        else {
            maxLen = 1;

```

```

    }
    left++;
    right++;
    globalMaxLen = Math.max(globalMaxLen, maxLen);
  }
  return globalMaxLen;
};

```

最长连续序列 【哈希表】

👉 【LeetCode 直通车】：128 最长连续序列（困难） [14]

题解

```

/**
 * @param {number[]} nums
 * @return {number}
 */
var longestConsecutive = function(nums) {
  if (nums.length === 0) return 0;
  const set = new Set(nums);
  const n = nums.length;
  let globalLongest = 1;
  for (let i = 0; i < n; i++) {
    if (!set.has(nums[i] - 1)) {
      let longest = 1;
      let currentNum = nums[i];
      while (set.has(currentNum + 1)) {
        currentNum += 1;
        longest++;
      }
      globalLongest = Math.max(globalLongest, longest);
    }
  }
  return globalLongest;
};

```

【面试真题】盛最多水的容器 【哈希表】

👉 【LeetCode 直通车】：11 盛最多水的容器（中等） [15]

题解

```

/**
 * @param {number[]} height
 * @return {number}
 */
var maxArea = function(height) {
    let n = height.length;
    let left = 0, right = n - 1;
    let maxOpacity = 0;
    while (left < right) {
        let res = Math.min(height[left], height[right]) * (right - left);
        maxOpacity = Math.max(maxOpacity, res);
        if (height[left] < height[right]) left++
        else right--;
    }
    return maxOpacity;
};

```

寻找两个正序数组的中位数【双指针】

👉 **【LeetCode 直通车】：4 寻找两个正序数组的中位数（困难） [16]**

题解

```

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var findMedianSortedArrays = function(nums1, nums2) {
    let m = nums1.length, n = nums2.length;
    let i = 0, j = 0;
    let newArr = [];
    while (i < m && j < n) {
        if (nums1[i] < nums2[j]) {
            newArr.push(nums1[i++]);
        } else {
            newArr.push(nums2[j++]);
        }
    }
    newArr = newArr.concat(i < m ? nums1.slice(i) : nums2.slice(j));

```

```

newArr = newArr.concat([nums1[select1] + nums2[select2]]),

const len = newArr.length;
console.log(newArr)
if (len % 2 === 0) {
    return (newArr[len / 2] + newArr[len / 2 - 1]) / 2;
} else {
    return newArr[Math.floor(len / 2)];
}
};

```

删除有序数组中的重复项【快慢指针】

👉 【LeetCode 直通车】：26 删除有序数组中的重复项（简单） [17]

题解

```

/**
 * @param {number[]} nums
 * @return {number}
 */
var removeDuplicates = function(nums) {
    if (nums.length <= 1) return nums.length;
    let lo = 0, hi = 0;
    while (hi < nums.length) {
        while (nums[lo] === nums[hi] && hi < nums.length) hi++;
        if (nums[lo] !== nums[hi] && hi < nums.length) {
            lo++;
            nums[lo] = nums[hi];
        }
        hi++;
    }
    return lo + 1;
};

```

👉 【LeetCode 直通车】：695 岛屿的最大面积（中等） [18]

题解

```

/**
 * @param {number[][]} grid
 * @return {number}
 */

```

```

    return number;
}

let maxX, maxY; let visited; let globalMaxArea;
var maxAreaOfIsland = function(grid) {
    visited = new Set();
    maxX = grid.length;
    maxY = grid[0].length;
    globalMaxArea = 0;
    for (let i = 0; i < maxX; i++) {
        for (let j = 0; j < maxY; j++) {
            if (grid[i][j] === 1) {
                visited.add(`${i}, ${j}`);
                globalMaxArea = Math.max(globalMaxArea, dfs(grid, i, j));
            }
            visited.clear();
        }
    }
    return globalMaxArea;
};

function dfs(grid, x, y) {
    let res = 1;
    for (let i = -1; i <= 1; i++) {
        for (let j = -1; j <= 1; j++) {
            if (Math.abs(i) === Math.abs(j)) continue;
            const newX = x + i;
            const newY = y + j;
            if (newX >= maxX || newX < 0 || newY >= maxY || newY < 0) continue;
            if (visited.has(`${newX}, ${newY}`)) continue;
            visited.add(`${newX}, ${newY}`);
            const areaCnt = grid[newX][newY];
            if (areaCnt === 1) {
                const cnt = dfs(grid, newX, newY);
                res += cnt;
            }
        }
    }
    return res;
}

```

和为K的子数组【哈希表】

👉 【LeetCode 直通车】：560 和为K的子数组（中等） [19]

题解

```

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var subarraySum = function(nums, k) {
    let cnt = 0;
    let sum0_i = 0, sum0_j = 0;
    let map = new Map();
    map.set(0, 1);
    for (let i = 0; i <= nums.length; i++) {
        sum0_i += nums[i];
        sum0_j = sum0_i - k;
        console.log('map', sum0_j, map.get(sum0_j))
        if (map.has(sum0_j)) {
            cnt += map.get(sum0_j);
        }
        let sumCnt = map.get(sum0_i) || 0;
        map.set(sum0_i, sumCnt + 1);
    }
    return cnt;
};

```

nSum问题【哈希表】 【系列】

- 👉 **【LeetCode 直通车】：1 两数之和（简单） [20]**
- 👉 **【LeetCode 直通车】：167 两数之和 II - 输入有序数组（简单） [21]**
- 👉 **【LeetCode 直通车】：15 三数之和（中等） [22]**
- 👉 **【LeetCode 直通车】：18 四数之和（中等） [23]**

受限于篇幅，这里只给出第一道题的代码模板，也是一面常考真题。

题解

```

/**
 * @param {number[]} nums
 * @param {number} target

```

```
* @return {number[]}
*/

var twoSum = function(nums, target) {
    let map2 = new Map();
    for (let i = 0; i < nums.length; i++) {
        map2.set(nums[i], i);
    }
    for (let i = 0; i < nums.length; i++) {
        if (map2.has(target - nums[i]) && map2.get(target - nums[i]) !== i) return [i, map2
    }
};
```

接雨水【暴力+备忘录优化】

👉 【LeetCode 直通车】：42 接雨水（困难） [24]

题解

```
/**
 * @param {number[]} height
 * @return {number}
 */
var trap = function(height) {
    let l_max = [], r_max = [];
    let len = height.length;
    let maxCapacity = 0;
    for (let i = 0; i < len; i++) {
        l_max[i] = height[i];
        r_max[i] = height[i];
    }
    for (let i = 1; i < len; i++) {
        l_max[i] = Math.max(l_max[i - 1], height[i]);
    }
    for (let j = len - 2; j >= 0; j--) {
        r_max[j] = Math.max(r_max[j + 1], height[j]);
    }
    for (let i = 0; i < len; i++) {
        maxCapacity += Math.min(l_max[i], r_max[i]) - height[i];
    }
    return maxCapacity;
};
```

跳跃游戏【贪心算法】【系列】

- 👉 **【LeetCode 直通车】：55 跳跃游戏（中等） [25]**
- 👉 **【LeetCode 直通车】：45 跳跃游戏 II（中等） [26]**

受限于篇幅，这里只给出第一道题的代码模板，也是一面常考真题。

题解

```
/**
 * @param {number[]} nums
 * @return {boolean}
 */
var canJump = function(nums) {
    let faster = 0;
    for (let i = 0; i < nums.length - 1; i++) {
        faster = Math.max(faster, i + nums[i]);
        if (faster <= i) return false;
    }
    return faster >= nums.length - 1;
};
```

高频算法题系列：二叉树

主要有以下几类高频考题：

- 二叉树的最近公共祖先【简单】【二叉树】
- 二叉搜索树中的搜索【简单】【二叉树】
- 删除二叉搜索树中的节点【中等】【二叉树】
- 完全二叉树的节点个数【中等】【二叉树】
- 二叉树的锯齿形层序遍历【中等】【二叉树】

二叉树的最近公共祖先【二叉树】

- 👉 **【LeetCode 直通车】：236 二叉树的最近公共祖先（简单） [27]**

题解

```
/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
/**
 * @param {TreeNode} root
 * @param {TreeNode} p
 * @param {TreeNode} q
 * @return {TreeNode}
 */
let visited; let parent;
var lowestCommonAncestor = function(root, p, q) {
    visited = new Set();
    parent = new Map();
    dfs(root);
    while (p !== null) {
        visited.add(p.val);
        p = parent.get(p.val);
    }
    while (q !== null) {
        if (visited.has(q.val)) {
            return q;
        }
        q = parent.get(q.val);
    }
    return null;
};
function dfs(root) {
    if (root.left !== null) {
        parent.set(root.left.val, root);
        dfs(root.left);
    }
    if (root.right !== null) {
        parent.set(root.right.val, root);
        dfs(root.right);
    }
}
```

二叉搜索树中的搜索【二叉树】

👉 【LeetCode 直通车】：700 二叉搜索树中的搜索（简单） [28]

题解

```
/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} val
 * @return {TreeNode}
 */
var searchBST = function(root, val) {
    if (root == null) return null;
    if (root.val === val) return root;
    if (root.val > val) {
        return searchBST(root.left, val);
    } else if (root.val < val) {
        return searchBST(root.right, val);
    }
};
```

删除二叉搜索树中的节点【二叉树】

👉 【LeetCode 直通车】：450 删除二叉搜索树中的节点（中等） [29]

题解

```
/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
```

```

    */
    /**
     * @param {TreeNode} root
     * @param {number} key
     * @return {TreeNode}
     */
    var deleteNode = function(root, key) {
        if (root == null) return null;
        if (root.val === key) {
            if (root.left == null && root.right == null) return null;
            if (root.left == null) return root.right;
            if (root.right == null) return root.left;
            if (root.left != null && root.right != null) {
                let target = getMinTreeMaxNode(root.left);
                root.val = target.val;
                root.left = deleteNode(root.left, target.val);
            }
        }
        if (root.val < key) {
            root.right = deleteNode(root.right, key);
        } else if (root.val > key) {
            root.left = deleteNode(root.left, key);
        }
        return root;
    };
    function getMinTreeMaxNode(root) {
        if (root.right == null) return root;
        return getMinTreeMaxNode(root.right);
    }
}

```

完全二叉树的节点个数【二叉树】

👉 【LeetCode 直通车】：222 完全二叉树的节点个数（中等） [30]

题解

```

    /**
     * Definition for a binary tree node.
     * function TreeNode(val) {
     *     this.val = val;
     *     this.left = this.right = null;
     * }

```

```

    */
    /**
     * @param {TreeNode} root
     * @return {number}
     */
    var countNodes = function(root) {
        if (root == null) return 0;
        let l = root, r = root;
        let lh = 0, rh = 0;
        while (l != null) {
            l = l.left;
            lh++;
        }
        while (r != null) {
            r = r.right;
            rh++;
        }
        if (lh === rh) {
            return Math.pow(2, lh) - 1;
        }
        return 1 + countNodes(root.left) + countNodes(root.right);
    };

```

二叉树的锯齿形层序遍历【二叉树】

👉 【LeetCode 直通车】：103 二叉树的锯齿形层序遍历（中等） [31]

题解

```

    /**
     * Definition for a binary tree node.
     * function TreeNode(val) {
     *     this.val = val;
     *     this.left = this.right = null;
     * }
     */
    /**
     * @param {TreeNode} root
     * @return {number[][]}
     */

```

```
let res;
var zigzagLevelOrder = function(root) {
    if (root == null) return [];
    res = [];
    BFS(root, true);
    return res;
};
function BFS(root, inOrder) {
    let arr = [];
    let resItem = [];
    let node;
    let stack1 = new Stack();
    let stack2 = new Stack();
    // 判断交换时机
    let flag;
    stack1.push(root);
    res.push([root.val]);
    inOrder = !inOrder;
    while (!stack1.isEmpty() || !stack2.isEmpty()) {
        if (stack1.isEmpty()) {
            flag = 'stack1';
        } else if (stack2.isEmpty()) {
            flag = 'stack2';
        }
        // 决定取那个栈里面的元素
        if (flag === 'stack2' && !stack1.isEmpty()) node = stack1.pop();
        else if (flag === 'stack1' && !stack2.isEmpty()) node = stack2.pop();
        if (inOrder) {
            if (node.left) {
                if (flag === 'stack1') {
                    stack1.push(node.left);
                } else {
                    stack2.push(node.left);
                }
                resItem.push(node.left.val);
            }
            if (node.right) {
                if (flag === 'stack1') {
                    stack1.push(node.right);
                } else {
                    stack2.push(node.right);
                }
                resItem.push(node.right.val);
            }
        }
    }
}
```

```

    } else {
        if (node.right) {
            if (flag === 'stack1') {
                stack1.push(node.right);
            } else {
                stack2.push(node.right);
            }
            resItem.push(node.right.val);
        }
        if (node.left) {
            if (flag === 'stack1') {
                stack1.push(node.left);
            } else {
                stack2.push(node.left);
            }
            resItem.push(node.left.val);
        }
    }
    // 判断下次翻转的时机
    if ((flag === 'stack2' && stack1.isEmpty()) || (flag === 'stack1' && stack2.isEmpty())) {
        inOrder = !inOrder;
        // 需要翻转了, 就加一轮值
        if (resItem.length > 0) {
            res.push(resItem);
        }
        resItem = [];
    }
}

class Stack {
    constructor() {
        this.count = 0;
        this.items = [];
    }
    push(element) {
        this.items[this.count] = element;
        this.count++;
    }
    pop() {
        if (this.isEmpty()) return undefined;
        const element = this.items[this.count - 1];
        delete this.items[this.count - 1];
        this.count--;
        return element;
    }
    size() {

```

```

        return this.count;
    }
    isEmpty() {
        return this.size() === 0;
    }
}

```

【🔥】高频算法题系列：排序算法

主要有以下几类高频考题：

- 用最少数量的箭引爆气球 【中等】 【排序】
- 合并区间 【中等】 【排序算法+区间问题】 【面试真题】

用最少数量的箭引爆气球 【排序算法】

👉 【LeetCode 直通车】：452 用最少数量的箭引爆气球（中等） [32]

题解

```

/**
 * @param {number[][]} points
 * @return {number}
 */
var findMinArrowShots = function(points) {
    if (points.length === 0) return 0;
    points.sort((a, b) => a[1] - b[1]);
    let cnt = 1;
    let resArr = [points[0]];
    let curr, last;
    for (let i = 1; i < points.length; i++) {
        curr = points[i];
        last = resArr[resArr.length - 1];
        if (curr[0] > last[1]) {
            resArr.push(curr);
            cnt++;
        }
    }
    return cnt;
}

```

};

合并区间【排序算法+区间问题】

👉 【LeetCode 直通车】：56 合并区间（中等） [33]

题解

```
/**
 * @param {number[][]} intervals
 * @return {number[][]}
 */
var merge = function(intervals) {
    if (intervals.length === 0) return [];
    intervals.sort((a, b) => a[0] - b[0]);
    let mergeArr = [intervals[0]];
    let last, curr;
    for (let j = 1; j < intervals.length; j++) {
        last = mergeArr[mergeArr.length - 1];
        curr = intervals[j];
        if (last[1] >= curr[0]) {
            last[1] = Math.max(curr[1], last[1]);
        } else {
            mergeArr.push(curr);
        }
    }
    return mergeArr;
};
```

高频算法题系列：二分查找

主要有以下几类高频考题：

- 寻找两个正序数组的中位数【困难】 【二分查找】
- 判断子序列【简单】 【二分查找】
- 在排序数组中查找元素的第一个和最后一个位置【中等】 【二分查找】

寻找两个正序数组的中位数【二分查找】

👉 【LeetCode 直通车】：4 寻找两个正序数组的中位数（困难） [34]

题解

```
/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var findMedianSortedArrays = function(nums1, nums2) {
    let m = nums1.length, n = nums2.length;
    let i = 0, j = 0;
    let newArr = [];
    while (i < m && j < n) {
        if (nums1[i] < nums2[j]) {
            newArr.push(nums1[i++]);
        } else {
            newArr.push(nums2[j++]);
        }
    }
    newArr = newArr.concat(i < m ? nums1.slice(i) : nums2.slice(j));
    const len = newArr.length;
    console.log(newArr)
    if (len % 2 === 0) {
        return (newArr[len / 2] + newArr[len / 2 - 1]) / 2;
    } else {
        return newArr[Math.floor(len / 2)];
    }
};
```

判断子序列【二分查找】

👉 【LeetCode 直通车】：392 判断子序列（简单） [35]

题解

```
/**
 * @param {string} s
 * @param {string} t
 * @return {boolean}
```

```

*/
var isSubsequence = function(s, t) {
    let hash = {};
    for (let i = 0; i < t.length; i++) {
        if (!hash[t[i]]) hash[t[i]] = [];
        hash[t[i]].push(i);
    }
    let lastMaxIndex = 0;
    for (let i = 0; i < s.length; i++) {
        if (hash[s[i]]) {
            const index = binarySearch(hash[s[i]], lastMaxIndex);
            console.log('index', index, hash[s[i]]);
            if (index === -1) return false;
            lastMaxIndex = hash[s[i]][index] + 1;
        } else return false;
    }
    return true;
};
function binarySearch(array, targetIndex) {
    let left = 0, right = array.length;
    while (left < right) {
        let mid = left + Math.floor((right - left) / 2);
        if (array[mid] >= targetIndex) {
            right = mid;
        } else if (array[mid] < targetIndex) {
            left = mid + 1;
        }
    }
    if (left >= array.length || array[left] < targetIndex) return -1;
    return left;
}

```

👤 在排序数组中查找元素的第一个和最后一个位置【二分搜索】

👉 【LeetCode 直通车】：34 在排序数组中查找元素的第一个和最后一个位置（中等） [36]

题解

```

/**
 * @param {number[]} nums

```

```

* @param {number} target
* @return {number[]}
*/
var searchRange = function(nums, target) {
    const left = leftBound(nums, target);
    const right = rightBound(nums, target);
    return [left, right];
};
function leftBound(nums, target) {
    let left = 0;
    let right = nums.length - 1;
    while (left <= right) {
        let mid = Math.floor(left + (right - left) / 2);
        if (nums[mid] === target) {
            right = mid - 1;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else if (nums[mid] > target) {
            right = mid - 1;
        }
    }
    if (left >= nums.length || nums[left] !== target) {
        return -1;
    }
    return left;
}
function rightBound(nums, target) {
    let left = 0;
    let right = nums.length - 1;
    while (left <= right) {
        let mid = Math.floor(left + (right - left) / 2);
        if (nums[mid] === target) {
            left = mid + 1;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else if (nums[mid] > target) {
            right = mid - 1;
        }
    }
    if (right < 0 || nums[right] !== target) {
        return -1;
    }
}
```

```
    return right;
}
```

【🔥】高频算法题系列：动态规划

主要有以下几类高频考题：

- 最长递增子序列【中等】【动态规划】
- 零钱兑换【中等】【动态规划】【面试真题】
- 最长公共子序列【中等】【动态规划】【面试真题】
- 编辑距离【困难】【动态规划】
- 最长回文子序列【中等】【动态规划】【面试真题】
- 最大子序和【简单】【动态规划】【面试真题】
- 买卖股票的最佳时机系列【系列】【动态规划】【面试真题】

最长递增子序列【动态规划】

👉 【LeetCode 直通车】：300 最长递增子序列（中等） [37]

题解

```
/**
 * @param {number[]} nums
 * @return {number}
 */
var lengthOfLIS = function(nums) {
    let maxLen = 0, n = nums.length;
    let dp = [];
    for (let i = 0; i < n; i++) {
        dp[i] = 1;
    }
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < i; j++) {
            if (nums[i] > nums[j]) {
                dp[i] = Math.max(dp[i], dp[j] + 1);
            }
        }
    }
}
```

```
        maxLen = Math.max(maxLen, dp[i]);
    }
    return maxLen;
};
```

【面试真题】零钱兑换【动态规划】

👉 【LeetCode 直通车】：322 零钱兑换（中等） [38]

题解

```
/**
 * @param {number[]} coins
 * @param {number} amount
 * @return {number}
 */
var coinChange = function(coins, amount) {
    if (amount === 0) return 0;
    let dp = [];
    for (let i = 0; i <= amount; i++) {
        dp[i] = amount + 1;
    }
    dp[0] = 0;
    for (let i = 0; i <= amount; i++) {
        for (let j = 0; j < coins.length; j++) {
            if (i >= coins[j]) {
                dp[i] = Math.min(dp[i - coins[j]] + 1, dp[i])
            }
        }
    }
    return dp[amount] === amount + 1 ? -1 : dp[amount];
};
```

【面试真题】最长公共子序列【动态规划】

👉 【LeetCode 直通车】：1143 最长公共子序列（中等） [39]

题解

```
/**
```

```

* @param {string} text1
* @param {string} text2
* @return {number}
*/
var longestCommonSubsequence = function(text1, text2) {
    let n1 = text1.length, n2 = text2.length;
    let dp = [];
    for (let i = -1; i < n1; i++) {
        dp[i] = [];
        for (let j = -1; j < n2; j++) {
            dp[i][j] = 0;
        }
    }
    for (let i = 0; i < n1; i++) {
        for (let j = 0; j < n2; j++) {
            if (text1[i] === text2[j]) {
                dp[i][j] = Math.max(dp[i][j], dp[i - 1][j - 1] + 1);
            } else {
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1])
            }
        }
    }
    return dp[n1 - 1][n2 - 1];
};

```

编辑距离【动态规划】

👉 【LeetCode 直通车】：72 编辑距离（困难） [40]

题解

```

/**
* @param {string} word1
* @param {string} word2
* @return {number}
*/
var minDistance = function(word1, word2) {
    let len1 = word1.length, len2 = word2.length;
    let dp = [];
    for (let i = 0; i <= len1; i++) {
        dp[i] = [];
    }
    for (let j = 0; j <= len2; j++) {
        dp[0][j] = j;
    }
    for (let i = 1; i <= len1; i++) {
        dp[i][0] = i;
        for (let j = 1; j <= len2; j++) {
            if (word1[i - 1] === word2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                dp[i][j] = Math.min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1] + 1) + 1;
            }
        }
    }
    return dp[len1][len2];
};

```

```

    for (let j = 0; j <= len2; j++) {
        dp[i][j] = 0;
        if (i === 0) {
            dp[i][j] = j;
        }
        if (j === 0) {
            dp[i][j] = i;
        }
    }
}
for (let i = 1; i <= len1; i++) {
    for (let j = 1; j <= len2; j++) {
        if (word1[i - 1] === word2[j - 1]) {
            dp[i][j] = dp[i - 1][j - 1];
        } else {
            dp[i][j] = Math.min(dp[i - 1][j] + 1, dp[i][j - 1] + 1, dp[i - 1][j - 1] + 1);
        }
    }
}
return dp[len1][len2];
};

```

【面试真题】最长回文子序列【动态规划】

👉 【LeetCode 直通车】：516 最长回文子序列（中等） [41]

题解

```

/**
 * @param {string} s
 * @return {number}
 */
var longestPalindromeSubseq = function(s) {
    let dp = [];
    for (let i = 0; i < s.length; i++) {
        dp[i] = [];
        for (let j = 0; j < s.length; j++) {
            dp[i][j] = 0;
        }
        dp[i][i] = 1;
    }
    for (let i = s.length - 1; i >= 0; i--) {

```

```

        for (let j = i + 1; j < s.length; j++) {
            if (s[i] === s[j]) {
                dp[i][j] = dp[i + 1][j - 1] + 2;
            } else {
                dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[0][s.length - 1];
};

```

【面试真题】👉 最大子序和【动态规划】

👉 【LeetCode 直通车】：53 最大子序和（简单） [42]

题解

```

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxSubArray = function(nums) {
    let maxSum = -Infinity;
    let dp = [], n = nums.length;
    for (let i = -1; i < n; i++) {
        dp[i] = 0;
    }
    for (let i = 0; i < n; i++) {
        dp[i] = Math.max(nums[i], dp[i - 1] + nums[i]);
        maxSum = Math.max(maxSum, dp[i]);
    }
    return maxSum;
};

```

【面试真题】👉 买卖股票的最佳时机【动态规划】

- 👉 【LeetCode 直通车】：121 买卖股票的最佳时机（简单） [43] 【面试真题】
- 👉 【LeetCode 直通车】：122 买卖股票的最佳时机 II（简单） [44]
- 👉 【LeetCode 直通车】：123 买卖股票的最佳时机 III（困难） [45]
- 👉 【LeetCode 直通车】：188 买卖股票的最佳时机IV（困难） [46]

- 📌 **【LeetCode 直通车】：309 买卖股票的最佳时机含冷冻期（中等） [47]**
- 📌 **【LeetCode 直通车】：714 买卖股票的最佳时机含手续费（中等） [48]**

受限于篇幅，这里只给出第一道题的代码模板，也是一面常考真题，笔者在面试字节跳动时就遇到过。

题解

```
/**
 * @param {number[]} prices
 * @return {number}
 */
var maxProfit = function(prices) {
    let dp = [];
    for (let i = -1; i < prices.length; i++) {
        dp[i] = []
        for (let j = 0; j <= 1; j++) {
            dp[i][j] = [];
            dp[i][j][0] = 0;
            dp[i][j][1] = 0;
            if (i === -1) {
                dp[i][j][1] = -Infinity;
            }
            if (j === 0) {
                dp[i][j][1] = -Infinity;
            }
            if (j === -1) {
                dp[i][j][1] = -Infinity;
            }
        }
    }
    for (let i = 0; i < prices.length; i++) {
        for (let j = 1; j <= 1; j++) {
            dp[i][j][0] = Math.max(dp[i - 1][j][0], dp[i - 1][j][1] + prices[i]);
            dp[i][j][1] = Math.max(dp[i - 1][j][1], dp[i - 1][j - 1][0] - prices[i]);
        }
    }
    return dp[prices.length - 1][1][0];
};
```

高频算法题系列：BFS

主要有以下几类高频考题：

- 打开转盘锁【中等】【BFS】
- 二叉树的最小深度【简单】【BFS】

| 打开转盘锁【BFS】

👉 【LeetCode 直通车】：752 打开转盘锁（中等） [49]

题解

```
/**
 * @param {string[]} deadends
 * @param {string} target
 * @return {number}
 */
var openLock = function(deadends, target) {
    let queue = new Queue();
    let visited = new Set();
    let step = 0;
    queue.push('0000');
    visited.add('0000');
    while (!queue.isEmpty()) {
        let size = queue.size();
        for (let i = 0; i < size; i++) {
            let str = queue.pop();
            if (deadends.includes(str)) continue;
            if (target === str) {
                return step;
            }
        }
        for (let j = 0; j < 4; j++) {
            let plusStr = plusOne(str, j);
            let minusStr = minusOne(str, j);
            if (!visited.has(plusStr)) {
                queue.push(plusStr);
                visited.add(plusStr);
            }
            if (!visited.has(minusStr)) {
```

```

        queue.push(minusStr);
        visited.add(minusStr)
    }
}
}
step++;
}
return -1;
};

function plusOne(str, index) {
    let strArr = str.split('');
    if (strArr[index] === '9') {
        strArr[index] = '0'
    } else {
        strArr[index] = (Number(strArr[index]) + 1).toString()
    }
    return strArr.join('');
}

function minusOne(str, index) {
    let strArr = str.split('');
    if (strArr[index] === '0') {
        strArr[index] = '9'
    } else {
        strArr[index] = (Number(strArr[index]) - 1).toString()
    }
    return strArr.join('');
}

class Queue {
    constructor() {
        this.items = [];
        this.count = 0;
        this.lowerCount = 0;
    }
    push(elem) {
        this.items[this.count++] = elem;
    }
    pop() {
        if (this.isEmpty()) {
            return;
        }
        const elem = this.items[this.lowerCount];
        delete this.items[this.lowerCount];
        this.lowerCount++;
        return elem;
    }
}

```

```
}
isEmpty() {
    if (this.size() === 0) return true;
    return false;
}
size() {
    return this.count - this.lowerCount;
}
}
```

二叉树的最小深度【BFS】

👉 【LeetCode 直通车】：111 二叉树的最小深度（简单） [50]

题解

```
/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var minDepth = function(root) {
    if (root == null) return 0;
    let depth = 1;
    let queue = new Queue();
    queue.push(root);
    while (!queue.isEmpty()) {
        let size = queue.size();
        for (let i = 0; i < size; i++) {
            const node = queue.pop();
            if (node.left == null && node.right == null) return depth;
            if (node.left) {
                queue.push(node.left);
            }
            if (node.right) {
                queue.push(node.right);
            }
        }
        depth++;
    }
}
```

```
    }  
  }  
  depth++;  
}  
return depth;  
};  
class Queue {  
  constructor() {  
    this.items = [];  
    this.count = 0;  
    this.lowerCount = 0;  
  }  
  push(elem) {  
    this.items[this.count++] = elem;  
  }  
  pop() {  
    if (this.isEmpty()) {  
      return;  
    }  
    const elem = this.items[this.lowerCount];  
    delete this.items[this.lowerCount];  
    this.lowerCount++;  
    return elem;  
  }  
  isEmpty() {  
    if (this.size() === 0) return true;  
    return false;  
  }  
  size() {  
    return this.count - this.lowerCount;  
  }  
}
```

【🔥】高频算法题系列：栈

主要有以下几类高频考题：

- 最小栈【简单】 【栈】
- 有效的括号【中等】 【栈】 【面试真题】
- 简化路径【中等】 【栈】
- 下一个更大元素 【系列】 【栈】

最小栈【栈】

👉 【LeetCode 直通车】：155 最小栈（简单） [51]

题解

```
/**
 * initialize your data structure here.
 */
var MinStack = function() {
    this.stack = [];
    this.minArr = [];
    this.count = 0;
    this.min = Number.MAX_SAFE_INTEGER;
};

/**
 * @param {number} x
 * @return {void}
 */
MinStack.prototype.push = function(x) {
    this.min = Math.min(this.min, x);
    this.minArr[this.count] = this.min;
    this.stack[this.count] = x;
    this.count++;
};

/**
 * @return {void}
 */
MinStack.prototype.pop = function() {
    const element = this.stack[this.count - 1];
    if (this.count - 2 >= 0) this.min = this.minArr[this.count - 2];
    else this.min = Number.MAX_SAFE_INTEGER;
    delete this.stack[this.count - 1];
    delete this.minArr[this.count - 1];
    this.count--;
    return element;
};

/**
 * @return {number}
 */
```

```

MinStack.prototype.top = function() {
    if (this.count >= 1) {
        return this.stack[this.count - 1];
    }
    return null;
};
/**
 * @return {number}
 */
MinStack.prototype.getMin = function() {
    const element = this.minArr[this.count - 1];
    return element;
};
/**
 * Your MinStack object will be instantiated and called as such:
 * var obj = new MinStack()
 * obj.push(x)
 * obj.pop()
 * var param_3 = obj.top()
 * var param_4 = obj.getMin()
 */

```

【系列】下一个更大元素 【栈】

- 👉 **【LeetCode 直通车】：496 下一个更大元素 I（简单） [52]**
- 👉 **【LeetCode 直通车】：503 下一个更大元素 II（中等） [53]**

受限于篇幅，这里只给出第一道题的代码模板

题解

```

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var nextGreaterElements = function(nums) {
    let ans = [];
    let stack = new Stack();
    const n = nums.length;
    for (let i = 2 * n - 1; i >= 0; i--) {

```

```
        while (!stack.isEmpty() && stack.top() <= nums[i % n]) {
            stack.pop();
        }
        ans[i % n] = stack.isEmpty() ? -1 : stack.top();
        stack.push(nums[i % n]);
    }
    return ans;
};

class Stack {
    constructor() {
        this.count = 0;
        this.items = [];
    }
    top() {
        if (this.isEmpty()) return undefined;
        return this.items[this.count - 1];
    }
    push(element) {
        this.items[this.count] = element;
        this.count++;
    }
    pop() {
        if (this.isEmpty()) return undefined;
        const element = this.items[this.count - 1];
        delete this.items[this.count - 1];
        this.count--;
        return element;
    }
    isEmpty() {
        return this.size() === 0;
    }
    size() {
        return this.count;
    }
}
```

【面试真题】有效的括号【栈】

👉 【LeetCode 直通车】：20 有效的括号（中等） [54]

题解

/**

```
* @param {string} s
* @return {boolean}
*/
var isValid = function(s) {
    if (s.length === 0) {
        return true;
    }
    if (s.length % 2 !== 0) {
        return false;
    }
    let map = {
        ')': '(',
        ']': '[',
        '}': '{',
    };
    let left = ['(', '[', '{'];
    let right = [')', ']', '}'];
    let stack = new Stack();
    for (let i = 0; i < s.length; i++) {
        if (!right.includes(s[i])) {
            stack.push(s[i]);
        } else {
            const matchStr = map[s[i]];
            while (!stack.isEmpty()) {
                const element = stack.pop();
                if (left.includes(element) && matchStr !== element) return false;
                if (element === matchStr) break;
            }
        }
    }
    return stack.isEmpty();
};

class Stack {
    constructor() {
        this.count = 0;
        this.items = [];
    }
    push(element) {
        this.items[this.count] = element;
        this.count++;
    }
    pop() {
        if (this.isEmpty()) return undefined;
    }
}
```

```

        const element = this.items[this.count - 1];
        delete this.items[this.count - 1];
        this.count--;
        return element;
    }
    isEmpty() {
        return this.size() === 0;
    }
    size() {
        return this.count;
    }
}

```

| 简化路径【栈】

👉 【LeetCode 直通车】：71 简化路径（中等） [55]

题解

```

/**
 * @param {string} path
 * @return {string}
 */
var simplifyPath = function(path) {
    let newPath = path.split('/');
    newPath = newPath.filter(item => item !== "");
    const stack = new Stack();
    for (let s of newPath) {
        if (s === '..') stack.pop();
        else if (s !== '.') stack.push(s);
    }
    if (stack.isEmpty()) return '/';
    let str = '';
    while (!stack.isEmpty()) {
        const element = stack.pop();
        str = '/' + element + str;
    }
    return str;
};
function handleBack(stack, tag, num) {
    if (!stack.isEmpty()) return num;
}

```

```
const element = stack.pop();
if (element === '..') return handleBack(stack, tag, num + 1);
else {
    stack.push(element);
    return num;
}
}
class Stack {
    constructor() {
        this.count = 0;
        this.items = [];
    }
    push(element) {
        this.items[this.count] = element;
        this.count++;
    }
    pop() {
        if (this.isEmpty()) return undefined;
        const element = this.items[this.count - 1];
        delete this.items[this.count - 1];
        this.count--;
        return element;
    }
    size() {
        return this.count;
    }
    isEmpty() {
        return this.size() === 0;
    }
}
```

【🔥】高频算法题系列：DFS

主要有以下几类高频考题：

- 岛屿的最大面积【中等】【DFS】
- 相同的树【简单】【DFS】

| 岛屿的最大面积【DFS】

👉 【LeetCode 直通车】：695 岛屿的最大面积（中等） [56]

题解

```
/**
 * @param {number[][]} grid
 * @return {number}
 */
let maxX, maxY; let visited; let globalMaxArea;
var maxAreaOfIsland = function(grid) {
    visited = new Set();
    maxX = grid.length;
    maxY = grid[0].length;
    globalMaxArea = 0;
    for (let i = 0; i < maxX; i++) {
        for (let j = 0; j < maxY; j++) {
            if (grid[i][j] === 1) {
                visited.add(`${i}, ${j}`);
                globalMaxArea = Math.max(globalMaxArea, dfs(grid, i, j));
            }
            visited.clear();
        }
    }
    return globalMaxArea;
};
function dfs(grid, x, y) {
    let res = 1;
    for (let i = -1; i <= 1; i++) {
        for (let j = -1; j <= 1; j++) {
            if (Math.abs(i) === Math.abs(j)) continue;
            const newX = x + i;
            const newY = y + j;
            if (newX >= maxX || newX < 0 || newY >= maxY || newY < 0) continue;
            if (visited.has(`${newX}, ${newY}`)) continue;
            visited.add(`${newX}, ${newY}`);
            const areaCnt = grid[newX][newY]
            if (areaCnt === 1) {
                const cnt = dfs(grid, newX, newY);
                res += cnt;
            }
        }
    }
    return res;
}
```

相同的树【DFS】

👉 【LeetCode 直通车】：100 相同的树（简单） [57]

题解

```
/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
/**
 * @param {TreeNode} p
 * @param {TreeNode} q
 * @return {boolean}
 */
var isSameTree = function(p, q) {
    if (p == null && q == null) return true;
    if (p == null || q == null) return false;
    if (p.val !== q.val) return false;
    return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
};
```

【🔥】高频算法题系列：回溯算法

主要有以下几类高频考题：

- N皇后【困难】 【回溯算法】 【面试真题】
- 全排列【中等】 【回溯算法】
- 括号生成【中等】 【回溯算法】
- 复原 IP 地址【中等】 【回溯算法】
- 子集【简单】 【回溯算法】

【面试真题】N皇后【回溯算法】

👉 【LeetCode 直通车】：51 N皇后（困难） [58]

题解

```
/**
 * @param {number} n
 * @return {string[][]}
 */
let result = [];var solveNQueens = function(n) {
    result = [];
    let board = [];
    for (let i = 0; i < n; i++) {
        board[i] = [];
        for (let j = 0; j < n; j++) {
            board[i][j] = '.'
        }
    }
    backtrack(0, board, n);
    return result;
};
function deepClone(board) {
    let res = [];
    for (let i = 0; i < board.length; i++) {
        res.push(board[i].join(''));
    }
    return res;
}
function backtrack(row, board, n) {
    if (row === n) {
        result.push(deepClone(board));
        return;
    }
    for (let j = 0; j < n; j++) {
        if (checkInvalid(board, row, j, n)) continue;
        board[row][j] = 'Q';
        backtrack(row + 1, board, n);
        board[row][j] = '.';
    }
}
function checkInvalid(board, row, column, n) {
```

```

// 行
for (let i = 0; i < n; i++) {
    if (board[i][column] === 'Q') return true;
}
for (let i = row - 1, j = column + 1; i >= 0 && j < n; i--, j++) {
    if (board[i][j] === 'Q') return true;
}
for (let i = row - 1, j = column - 1; i >= 0 && j >= 0; i--, j--) {
    if (board[i][j] === 'Q') return true;
}
return false;
}

```

全排列【回溯算法】

👉 **【LeetCode 直通车】：46 全排列（中等） [59]**

题解

```

/**
 * @param {number[]} nums
 * @return {number[][]}
 */
let results = [];var permute = function(nums) {
    results = [];
    backtrack(nums, []);
    return results;
};
function backtrack(nums, track) {
    if (nums.length === track.length) {
        results.push(track.slice());
        return;
    }
    for (let i = 0; i < nums.length; i++) {
        if (track.includes(nums[i])) continue;
        track.push(nums[i]);
        backtrack(nums, track);
        track.pop();
    }
}

```

括号生成【回溯算法】

 【LeetCode 直通车】：22 括号生成（中等） [60]**题解**

```
/**
 * @param {number} n
 * @return {string[]}
 */
var generateParenthesis = function(n) {
    let validRes = [];
    backtrack(n * 2, validRes, '');
    return validRes;
};
function backtrack(len, validRes, bracket) {
    if (bracket.length === len) {
        if (isValidCombination(bracket)) {
            validRes.push(bracket);
        }
        return;
    }
    for (let str of ['(', ')']) {
        bracket += str;
        backtrack(len, validRes, bracket);
        bracket = bracket.slice(0, bracket.length - 1);
    }
}
function isValidCombination(bracket) {
    let stack = new Stack();
    for (let i = 0; i < bracket.length; i++) {
        const str = bracket[i];
        if (str === '(') {
            stack.push(str);
        } else if (str === ')') {
            const top = stack.pop();
            if (top !== '(') return false;
        }
    }
    return stack.isEmpty();
}
class Stack {
    constructor() {
        this.count = 0;
    }
}
```

```

        this.items = [];
    }
    push(element) {
        this.items[this.count] = element;
        this.count++;
    }
    pop() {
        if (this.isEmpty()) return;
        const element = this.items[this.count - 1];
        delete this.items[this.count - 1];
        this.count--;
        return element;
    }
    size() {
        return this.count;
    }
    isEmpty() {
        return this.size() === 0;
    }
}

```

复原 IP 地址【回溯算法】

👉 **【LeetCode 直通车】：93 复原 IP 地址（中等） [61]**

题解

```

/**
 * @param {string} s
 * @return {string[]}
 */
var restoreIpAddresses = function(s) {
    if (s.length > 12) return [];
    let res = [];
    const track = [];
    backtrack(s, track, res);
    return res;
};
function backtrack(s, track, res) {
    if (track.length === 4 && s.length === 0) {
        res.push(track.join('.'));
        return;
    }
}

```

```

    }
    let len = s.length >= 3 ? 3 : s.length;
    for (let i = 0; i < len; i++) {
        const c = s.slice(0, i + 1);
        if (parseInt(c) > 255) continue;
        if (i >= 1 && parseInt(c) < parseInt((1 + '0'.repeat(i)))) continue;
        track.push(c);
        backtrack(s.slice(i + 1), track, res);
        track.pop();
    }
}

```

子集【回溯算法】

👉 **【LeetCode 直通车】：78 子集（中等） [62]**

题解

```

/**
 * @param {number[]} nums
 * @return {number[][]}
 */
var subsets = function(nums) {
    if (nums.length === 0) return [[]];
    let resArr = [];
    backtrack(nums, 0, [], resArr);
    return resArr;
};
function backtrack(nums, index, subArr, resArr) {
    if (Array.isArray(subArr)) {
        resArr.push(subArr.slice());
    }
    if (index === nums.length) {
        return;
    }
    for (let i = index; i < nums.length; i++) {
        subArr.push(nums[i]);
        backtrack(nums, i + 1, subArr, resArr);
        subArr.pop(nums[i]);
    }
}

```

文末福利

推荐一个非常有帮助的刷算法题的网址，**labuladong 的算法小抄** [63]，通过套路，认准高频题目，直通大厂；这本小炒目前已经出版成书，对应的 **Github 仓库** [64] 也有 86.2K Star，而且作者还在频繁更新，非常值得学习！

❤️ 谢谢

往期精文

- **字节跳动最爱考的前端面试题:JavaScript 基础** [65] 2696 🍊
- **字节跳动最爱考的前端面试题:CSS 基础** [66] 687 🍊
- **字节跳动最爱考的前端面试题:计算机网络基础** [67] 761 🍊

欢迎关注公众号：**图雀社区**。如果你想从零开始以实战的方式学习一门技术，亦或是想动手做一个比较完整的项目以准备面试，相信「图雀社区」的内容都能够帮助到你，成为初入前端的你成长路上的指南针。

原创不易

喜欢的话原创不易，给点鼓励吧 ❤️ 别忘了 分享、点赞、在看 三连哦~。

参考资料

- [1] 【LeetCode 直通车】：234 回文链表（简单）：<https://leetcode-cn.com/problems/palindrome-linked-list/>
- [2] 【LeetCode 直通车】：206 反转链表（简单）：<https://leetcode-cn.com/problems/reverse-linked-list/>
- [3] 【LeetCode 直通车】：23 合并K个升序链表（困难）：<https://leetcode-cn.com/problems/merge-k-sorted-lists/>
- [4] 【LeetCode 直通车】：25 K 个一组翻转链表（困难）：<https://leetcode-cn.com/problems/reverse-nodes-in-k-group/>
- [5] 【LeetCode 直通车】：141 环形链表（简单）：<https://leetcode-cn.com/problems/linked-list-cycle/>
- [6] 【LeetCode 直通车】：148 排序链表（中等）：<https://leetcode-cn.com/problems/sort-list/>

- [7] 【LeetCode 直通车】：160 相交链表（简单）：<https://leetcode-cn.com/problems/intersection-of-two-linked-lists/>
- [8] 【LeetCode 直通车】：5 最长回文子串（中等）：<https://leetcode-cn.com/problems/longest-palindromic-substring/>
- [9] 【LeetCode 直通车】：14 最长公共前缀（简单）：<https://leetcode-cn.com/problems/longest-common-prefix/>
- [10] 【LeetCode 直通车】：3 无重复字符的最长子串（中等）：<https://leetcode-cn.com/problems/longest-substring-without-repeating-characters/>
- [11] 【LeetCode 直通车】：76 最小覆盖子串（困难）：<https://leetcode-cn.com/problems/minimum-window-substring/>
- [12] 【LeetCode 直通车】：354 俄罗斯套娃信封问题（困难）：<https://leetcode-cn.com/problems/russian-doll-envelopes/>
- [13] 【LeetCode 直通车】：674 最长连续递增序列（简单）：<https://leetcode-cn.com/problems/longest-continuous-increasing-subsequence/>
- [14] 【LeetCode 直通车】：128 最长连续序列（困难）：<https://leetcode-cn.com/problems/longest-consecutive-sequence/>
- [15] 【LeetCode 直通车】：11 盛最多水的容器（中等）：<https://leetcode-cn.com/problems/container-with-most-water/>
- [16] 【LeetCode 直通车】：4 寻找两个正序数组的中位数（困难）：<https://leetcode-cn.com/problems/median-of-two-sorted-arrays/>
- [17] 【LeetCode 直通车】：26 删除有序数组中的重复项（简单）：<https://leetcode-cn.com/problems/remove-duplicates-from-sorted-array/>
- [18] 【LeetCode 直通车】：695 岛屿的最大面积（中等）：<https://leetcode-cn.com/problems/max-area-of-island/>
- [19] 【LeetCode 直通车】：560 和为K的子数组（中等）：<https://leetcode-cn.com/problems/subarray-sum-equals-k/>
- [20] 【LeetCode 直通车】：1 两数之和（简单）：<https://leetcode-cn.com/problems/two-sum/>
- [21] 【LeetCode 直通车】：167 两数之和 II - 输入有序数组（简单）：<https://leetcode-cn.com/problems/two-sum-ii-input-array-is-sorted/>
- [22] 【LeetCode 直通车】：15 三数之和（中等）：<https://leetcode-cn.com/problems/3sum/>
- [23] 【LeetCode 直通车】：18 四数之和（中等）：<https://leetcode-cn.com/problems/4sum/>
- [24] 【LeetCode 直通车】：42 接雨水（困难）：<https://leetcode-cn.com/problems/trapping-rain-water/>
- [25] 【LeetCode 直通车】：55 跳跃游戏（中等）：<https://leetcode-cn.com/problems/jump-game/>
- [26] 【LeetCode 直通车】：45 跳跃游戏 II（中等）：<https://leetcode-cn.com/problems/jump-game-ii/>
- [27] 【LeetCode 直通车】：236 二叉树的最近公共祖先（简单）：<https://leetcode-cn.com/problems/lowest-common-ancestor-of-a-binary-tree/>
- [28] 【LeetCode 直通车】：700 二叉搜索树中的搜索（简单）：<https://leetcode-cn.com/problems/search-in-a-binary-search-tree/>

ms/search-in-a-binary-search-tree/

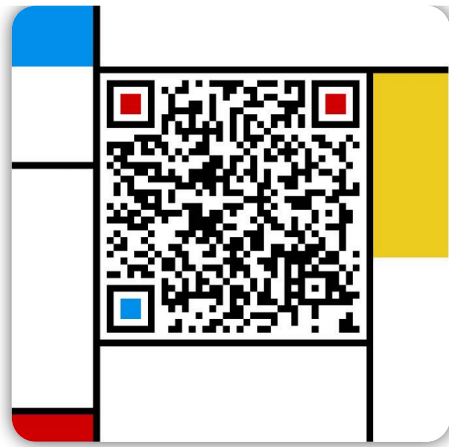
- [29] 【LeetCode 直通车】：450 删除二叉搜索树中的节点（中等）：<https://leetcode-cn.com/problems/delete-node-in-a-bst/>
- [30] 【LeetCode 直通车】：222 完全二叉树的节点个数（中等）：<https://leetcode-cn.com/problems/count-complete-tree-nodes/>
- [31] 【LeetCode 直通车】：103 二叉树的锯齿形层序遍历（中等）：<https://leetcode-cn.com/problems/binary-tree-zigzag-level-order-traversal/>
- [32] 【LeetCode 直通车】：452 用最少数量的箭引爆气球（中等）：<https://leetcode-cn.com/problems/minimum-number-of-arrows-to-burst-balloons/>
- [33] 【LeetCode 直通车】：56 合并区间（中等）：<https://leetcode-cn.com/problems/merge-intervals/>
- [34] 【LeetCode 直通车】：4 寻找两个正序数组的中位数（困难）：<https://leetcode-cn.com/problems/median-of-two-sorted-arrays/>
- [35] 【LeetCode 直通车】：392 判断子序列（简单）：<https://leetcode-cn.com/problems/is-subsequence/>
- [36] 【LeetCode 直通车】：34 在排序数组中查找元素的第一个和最后一个位置（中等）：<https://leetcode-cn.com/problems/find-first-and-last-position-of-element-in-sorted-array/>
- [37] 【LeetCode 直通车】：300 最长递增子序列（中等）：<https://leetcode-cn.com/problems/longest-increasing-subsequence/>
- [38] 【LeetCode 直通车】：322 零钱兑换（中等）：<https://leetcode-cn.com/problems/coin-change/>
- [39] 【LeetCode 直通车】：1143 最长公共子序列（中等）：<https://leetcode-cn.com/problems/longest-common-subsequence/>
- [40] 【LeetCode 直通车】：72 编辑距离（困难）：<https://leetcode-cn.com/problems/edit-distance/>
- [41] 【LeetCode 直通车】：516 最长回文子序列（中等）：<https://leetcode-cn.com/problems/longest-palindromic-subsequence/>
- [42] 【LeetCode 直通车】：53 最大子序和（简单）：<https://leetcode-cn.com/problems/maximum-subarray/>
- [43] 【LeetCode 直通车】：121 买卖股票的最佳时机（简单）：<https://leetcode-cn.com/problems/best-time-to-buy-and-sell-stock/>
- [44] 【LeetCode 直通车】：122 买卖股票的最佳时机 II（简单）：<https://leetcode-cn.com/problems/best-time-to-buy-and-sell-stock-ii/>
- [45] 【LeetCode 直通车】：123 买卖股票的最佳时机 III（困难）：<https://leetcode-cn.com/problems/best-time-to-buy-and-sell-stock-iii/>
- [46] 【LeetCode 直通车】：188 买卖股票的最佳时机 IV（困难）：<https://leetcode-cn.com/problems/best-time-to-buy-and-sell-stock-iv/>
- [47] 【LeetCode 直通车】：309 买卖股票的最佳时机含冷冻期（中等）：<https://leetcode-cn.com/problems/best-time-to-buy-and-sell-stock-with-cooldown/>
- [48] 【LeetCode 直通车】：714 买卖股票的最佳时机含手续费（中等）：<https://leetcode-cn.com/problems/best-time-to-buy-and-sell-stock-with-transaction-fee/>

problems/best-time-to-buy-and-sell-stock-with-transaction-fee/

- [49] 【LeetCode 直通车】：752 打开转盘锁（中等）：<https://leetcode-cn.com/problems/open-the-lock/>
- [50] 【LeetCode 直通车】：111 二叉树的最小深度（简单）：<https://leetcode-cn.com/problems/minimum-depth-of-binary-tree/>
- [51] 【LeetCode 直通车】：155 最小栈（简单）：<https://leetcode-cn.com/problems/min-stack/submissions/>
- [52] 【LeetCode 直通车】：496 下一个更大元素 I（简单）：<https://leetcode-cn.com/problems/next-greater-element-i/>
- [53] 【LeetCode 直通车】：503 下一个更大元素 II（中等）：<https://leetcode-cn.com/problems/next-greater-element-ii/>
- [54] 【LeetCode 直通车】：20 有效的括号（中等）：<https://leetcode-cn.com/problems/valid-parentheses/>
- [55] 【LeetCode 直通车】：71 简化路径（中等）：<https://leetcode-cn.com/problems/simplify-path/>
- [56] 【LeetCode 直通车】：695 岛屿的最大面积（中等）：<https://leetcode-cn.com/problems/max-area-of-island/>
- [57] 【LeetCode 直通车】：100 相同的树（简单）：<https://leetcode-cn.com/problems/same-tree/>
- [58] 【LeetCode 直通车】：51 N皇后（困难）：<https://leetcode-cn.com/problems/n-queens/>
- [59] 【LeetCode 直通车】：46 全排列（中等）：<https://leetcode-cn.com/problems/permutations/>
- [60] 【LeetCode 直通车】：22 括号生成（中等）：<https://leetcode-cn.com/problems/generate-parentheses/>
- [61] 【LeetCode 直通车】：93 复原 IP 地址（中等）：<https://leetcode-cn.com/problems/restore-ip-addresses/>
- [62] 【LeetCode 直通车】：78 子集（中等）：<https://leetcode-cn.com/problems/subsets/>
- [63] labuladong 的算法小抄：<https://www.yuque.com/tuturu/interview/labuladong>：[https](https://github.com/labuladong/fucking-algorithm)
- [64] Github 仓库：<https://github.com/labuladong/fucking-algorithm>
- [65] 字节跳动最爱考的前端面试题:JavaScript 基础：<https://juejin.cn/post/6934500357091360781>
- [66] 字节跳动最爱考的前端面试题:CSS 基础：<https://juejin.cn/post/6936913689115099143>
- [67] 字节跳动最爱考的前端面试题:计算机网络基础：<https://juejin.cn/post/6939691851746279437>

关于我

下面的是我的个人二维码图片，希望能跟大家一起进阶，共同进步。



个人二维码

建立了一个技术群，如果你想了解到更多关于IT行业的技术以及生活中遇到的问题，欢迎小伙伴进群交流，只需添加我好友，备注：[进群](#)即可，期待你们的加入。

点击公众号，[星标置顶](#)，每一次分享都不会错过！

**fivecode**

专注分享java,大数据等相关知识

公众号

往期TOP5文章

[考研：谁不是一边在努力，一边想放弃？（师弟2020湖大考研经历）](#)

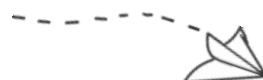
[练习C编程这一篇就够了](#)

[学习C基础这一篇就够了](#)

[周六失眠... 连夜整理了68道C语言/C++常见面试题\(含答案\)](#)

[意中发现一位大佬的算法刷题pdf笔](#)

喜欢本文，点个“[在看](#)”告诉我



喜欢此内容的人还喜欢

C# 开源一个新的雪花算法
DotNet

Vue 3.0 diff 算法及原理
前端达人

回溯算法详解（修订版）
码农有道